A Guide to Using Coding Agents for Open-Source Code Structure Analysis – Focusing on mini-swe-agent

Prompt design for coding agents · Placing `AGENTS.md` in Antigravity · Filename case-sensitivity caveats

Date: 2026-10-12

Q. After installing an open-source codebase, what requests (prompts) should you give a coding agent to understand how that code works, from a software engineering (SWE) perspective?

1. Prompt Design for Understanding How Open-Source Code Works

Rather than asking "explain this code" in one shot, you get far better results by breaking the request into stages. At every stage, require **evidence (file:line)**, **actual execution checks**, and **saving the results to files**. This makes the agent actually read and run the code instead of guessing plausibly.

Preparation 1: Downloading, Installing, and Using the Antigravity IDE

■ **Antigravity IDE overview:** Antigravity IDE is a next-generation, AI-agent-based software development environment (IDE) developed by Google DeepMind. Going beyond simple code completion, it is designed for pair programming with a coding agent that autonomously uses powerful tools such as file exploration, source code analysis, terminal command execution, and web search.

■ **Download and installation:** 1) Visit the official Antigravity distribution website (or your organization's portal) and download the installer for your operating system (Windows / macOS / Linux).
2) Run the downloaded installer (e.g., the Setup executable on Windows) and follow the instructions to complete the installation.
3) When you launch the IDE for the first time, a Google account sign-in and initial setup wizard will run; once authentication is complete, the main screen becomes active.

■ **Main interface layout of the Antigravity IDE:** To make collaboration with the agent convenient, the Antigravity IDE is organized into three core areas:
• Code editor & file explorer (left/center): view the project file tree, read and edit source code
• Integrated terminal (bottom/Terminal): check the status of builds, tests, and CLI commands
• ★ Agent Manager chat (right-hand chat panel): the main channel for talking with the coding agent in real time and directing analysis and development work

■ **How to use the Agent Manager chat:** 1) Start a chat session: click the '+ New Conversation' button at the top right to create a new work session.
2) Enter a natural-language prompt: type your analysis goal or task requirements in natural language in the input box at the bottom and send it.
3) Let the agent run autonomously and observe: the agent reads files according to your instructions, autonomously calls the relevant tools, and reports results in the chat.
4) Approve tool/command execution (Human-in-the-Loop): when the agent proposes running a shell command or modifying a file, an approval card appears in the chat. After reviewing it, click 'Approve' or 'Always Allow' to keep execution safely under your control.

Preparation 2: Cloning the mini-swe-agent Open-Source Repository and Opening the Workspace (mini-swe-agent)

■ **Step 1 - Clone the target repository locally:** To begin the analysis, clone the `mini-swe-agent` open-source repository, the analysis target, to your local machine. Open a terminal (PowerShell or Bash) and run the following commands:

```
# 1. Clone the mini-swe-agent open-source repository
git clone https://github.com/SWE-agent/mini-swe-agent.git

# 2. Move into the cloned repository directory
cd mini-swe-agent
```

■ **Step 2 - Open the 'mini-swe-agent' folder in the Antigravity IDE:** Once the clone is complete, launch the Antigravity IDE and prepare the workspace as follows:

1. Open the folder: click File > Open Folder... in the top menu (shortcut: Windows Ctrl+K, Ctrl+O / Mac Cmd+O), then select and open the `mini-swe-agent` folder you just cloned (or your local mini-swe-agent directory).
2. Check the project root: confirm that the mini-swe-agent folder structure (`src/minisweagent` , `tests` , `docs` , `pyproject.toml` , etc.) appears correctly in the Explorer on the left.

`mini-swe-agent` folder in the Antigravity IDE, carry out the detailed analysis in the Agent Manager chat on the right:

1. Review the working rules (`AGENTS.md`): check the existing `AGENTS.md` file in the project root directory (which defines mini-swe-agent's 5 core layers and coding/testing guidelines), and ensure the analysis principles are respected (no source code modification during analysis, cite path:line, distinguish [Verified]/[Inferred]).
2. Activate the Agent Manager chat: start a '+ New Conversation' in the Agent Manager panel on the right.
3. Proceed with the interactive, step-by-step analysis: in the message box of the Agent Manager chat, enter the prompts below one at a time, in order, from the 'Step 0 prompt' through the 'Step 8 prompt and bonus', asking the coding agent to carry out the analysis. The agent follows the rules strictly while analyzing files, building/testing, and drawing architecture diagrams, and reports the results.

Step 0: Give the Working Rules First

Give these once at the start, or put them in `CLAUDE.md` / `AGENTS.md` .

```
I want to analyze how this repository (mini-swe-agent) works. Follow these rules.
- Do not modify the source code during the analysis phase (read and run only).
- Back every claim with evidence in `path:line` format.
- Distinguish what you verified in the code from what you inferred, using [Verified]/[Inferred].
- Save results cumulatively as Markdown files under docs/analysis/.
```

Step 1: Get the Lay of the Land (Orientation)

```
Step 1: Skim the whole repository, summarize the following, and save it to
docs/analysis/01_overview.md.
1. The problem this project solves (flag any differences between the README and the actual code)
2. The role of each top-level directory (src/minisweagent, tests, docs, etc. - one line each)
3. Entry points: CLI entry points (mini, mini-swe-agent), run scripts (run.mini, run.hello_world), etc.
4. Build, environment setup, and test commands (uv, pip, pytest), plus primary external dependencies
(litellm, typer, jinja2, etc.)
5. The top 5–7 core modules (Agent, Environment, Model, Run, etc.) and why you judged them to be core
```

Step 2: Actually Run It (Dynamic Check)

```
Step 2: Build the virtual environment (.venv) and run unit tests with pytest. Then run one of the simplest execution examples (e.g., python -m minisweagent.run.hello_world or mini --help). Record the commands you ran, the results, and the causes of any failures, and correct anything that was wrong in the Step 1 analysis. Save these execution results to 'docs/analysis/02_build_test_execution.md'.
```

Static analysis alone misses places where the documentation and the actual behavior diverge.

Step 3: Install Missing Tools and Make Failed Runs Succeed (Fix Failures)

```
Step 3: Based on the Step 2 results in 'docs/analysis/02_build_test_execution.md', resolve any missing dependency packages or environment configurations (such as .env API keys) to make the tests and runs succeed. Run the tests and the hello_world example again in the updated environment, verify the results, and record them.
```

Step 4: Architecture View (Structure)

```
Step 4: Analyze the dependencies and interface structure between mini-swe-agent modules.
- Layer structure (interfaces/protocols in __init__.py → agents, environments, models → run) and dependency direction
- Polymorphic design across Agent, Environment, and Model protocols
- Any circular dependencies or layer violations
- Draw a component diagram in Mermaid (at the C4 Container/Component level)
Save the results to 'docs/analysis/04_SW_structure.md'.
```

Step 5: Trace One Core Flow End to End (End-to-End Trace)

This is the most valuable step of the analysis.

```
Step 5: Trace the representative use case "when a user executes a mini-swe-agent run script
(run.hello_world or run.mini)" from the entry point to the final output.
- The call chain in order: CLI argument parsing → configuration merge → Model/Environment/Agent
instantiation → Agent.run() loop (prompt LLM → execute shell action → return observation) → Trajectory
output (function name + path:line)
- Data and state changes across steps (AgentState, Trajectory, etc.)
- Exception handling and error recovery paths (cost limit exceeded, step limit reached, syntax error,
etc.)
- Draw a Mermaid sequence diagram
Then verify the trace against actual execution logs.
(Revert any temporary verification code when finished.)
```

Step 6: Data Model, Configuration, and Extension Points

```
Step 6: Summarize the data model, configuration, and extension points of mini-swe-agent.
1. Core data structures/entities (AgentConfig, EnvironmentConfig, ModelConfig, Trajectory, Message,
etc.) and their lifecycle
2. Order and precedence of configuration loading (mini.yaml default → user-specified config YAML →
environment variables / .env → CLI flags)
3. Extension points: custom Model adapters, new Environments (Local, Docker, Singularity, Swerex),
custom Agent subclasses
4. Design patterns visible in the code (Protocol-based Strategy pattern, Factory pattern, Dataclass
configuration management, etc.) and where they are used
```

Step 7: Lock In Your Understanding with Tests (Characterization Test)

```
Step 7:
Write characterization tests using pytest that verify 3 core behaviors of mini-swe-agent analyzed so
far (e.g., model interface invocation, environment command execution, single agent step loop) exactly
as they currently behave. If these tests pass, our understanding is correct.
```

Step 8: Wrap-Up and Open Questions

```
Step 8 (final step):
Combine the contents of docs/analysis/ and write ARCHITECTURE.md
(one-page summary + diagrams + core flow + mini-swe-agent's core philosophy: simplicity and
readability).
At the end, include a list of "things not yet verified / open questions".
```

Bonus: Deep Dive into the Coding Agent (mini-swe-agent)

mini-swe-agent is a minimal, highly readable open-source SWE coding agent implementation. To dissect this coding agent from the "model + harness" perspective, ask the following:

```
Dissect mini-swe-agent from the "model + harness" perspective. For each item below, explain where it is
implemented (path:line) and how it works.
1. Agent loop: the flow of one turn in Agent.run() and step() (LLM call → action/tool parsing →
environment execution → observation return) and termination conditions (COMPLETE, step/cost limits)
2. Tool & execution interface: Environment protocol (execute method), local/container sandboxing,
timeout, and output capture mechanisms
3. Context and prompt management: Jinja2 templates (system, instance prompts) rendering and
conversation history management
4. Control mechanisms: cost_limit, step_limit, interactive confirmation mode (human-in-the-loop)
5. Model interface: LiteLLM-based Model protocol abstraction, multi-provider support, retry logic
(tenacity)
6. Extensibility and polymorphism: Protocol-based Agent / Environment / Model plugin structure
Finally, create a comparison table highlighting the trade-offs and design principles of mini-swe-agent
(ultra-minimal agent) compared to full-featured frameworks like SWE-agent or OpenHands.
```

Operational Tips

- **One stage per session:** Proceed one step at a time. Saving results to files means the next session can simply read those files and pick up where you left off, preventing broken analysis even when context fills up.
- **Split large repos in parallel:** If sub-agents are available, ask the agent to "split exploration by directory in parallel, then synthesize".
- **Cross-validate results:** Do not blindly trust explanations from the agent; always include steps like Step 2 and Step 5 that verify behavior through direct execution.

2. Where to Place AGENTS.md in Antigravity IDE

In Antigravity, place it at the **very top of the opened folder (the workspace root)**. According to Antigravity's official documentation, an `AGENTS.md` at that location is read as a workspace rule that always applies without needing frontmatter.

```
mini-swe-agent/              ← Folder opened with "Open Folder" in Antigravity
├── AGENTS.md               ← Place here (applies to whole repo; architecture overview & coding rules)
├── README.md
├── pyproject.toml
├── src/
│   └── minisweagent/       ← mini-swe-agent source code directory
│       ├── __init__.py     ← Base interface / protocol definitions
│       ├── agents/         ← Agent control flow
│       ├── environments/   ← Environment execution logic
│       ├── models/         ← LLM interfaces
│       └── run/            ← Execution entrypoint scripts
├── tests/
└── docs/analysis/          ← Analysis output directory
```

Scope by Location

Location	Scope
Workspace root `AGENTS.md`	Entire repository (mini-swe-agent already has this file defining coding style, design principles, and test guidelines)
Subfolder `AGENTS.md`	Applies only when working on files in that folder. Traverses upward from file folder to root; closer folders take precedence
`~/.gemini/AGENTS.md`	`C:\Users\<user>\.gemini\AGENTS.md`
`.agents/rules/*.md`	Used when splitting rules into multiple files. Requires YAML frontmatter; rules subfolders are not read

Things to Watch Out For

- **Verify opened folder:** If you open a parent folder above the repository, that parent becomes the workspace root. Always open the repository folder itself (`mini-swe-agent`).
- **Start in a new conversation:** After creating or updating rules, start a fresh conversation. Continuing an old conversation may not pick up rule changes.
- **Confirm active rules:** Ask "Summarize the rules currently being applied" to verify.
- **If rules are not recognized:** In some environments, if `AGENTS.md` is not picked up directly, create a rule file under `.agents/rules/` that references `AGENTS.md` in the root.
- **Clean repo management:** Adding `docs/analysis/` to `.git/info/exclude` prevents local analysis artifacts from showing up in Git commits.

3. Filename Case Sensitivity: AGENTS.md vs agents.md

Always name the file `AGENTS.md` (uppercase). The standard filename is uppercase, and some tools are case-sensitive. Whether `agents.md` is recognized depends on the OS and file-lookup method.

Environment	**Is `agents.md` also recognized?**
Windows (NTFS)	Usually recognized, because the filesystem is case-insensitive
macOS (Default APFS)	Usually recognized, for the same reason as Windows
Linux, WSL, Docker	Not recognized. Treated as completely different files

Cases Where Problems Can Occur Even on Windows

- **Directory-listing comparison:** If a tool iterates over directory contents and compares filenames as strings instead of directly opening the path, `agents.md` is treated as different from `AGENTS.md` and skipped.
- **Moving across environments:** Committing to Git, or opening the repo on Linux/WSL/CI, will subject the file to Linux rules, causing it to be ignored.
- **Clashing filenames:** Windows cannot hold both files in the same directory. If a repo created on Linux has both, cloning on Windows will cause one to overwrite the other.

If You Already Created agents.md

On Windows, simply changing letter case might not register immediately with the filesystem. Rename in two steps:

```
ren agents.md tmp.md
ren tmp.md AGENTS.md
```

If tracked by Git, run `git mv agents.md AGENTS.md` in one line.

References

- mini-swe-agent GitHub Repository
- mini-swe-agent Official Documentation
- Antigravity Docs: Rules
- Agentpedia: Antigravity AGENTS.md not working fix
- The Prompt Shelf: Antigravity AGENTS.md rules guide